Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE-3191 Base de Datos

SmartCity Analytics

DOCUMENTACIÓN TÉCNICA — SPRINT 1

Integrantes:

Daniel Burger — 2024145222

José Pablo Alpízar — 2024088025

Fernanda Vargas — 2022008164

Kembly Garro — 2024134020

Cartago, Costa Rica

Octubre, 2026

1. Descripción del problema

Las ciudades generan constantemente grandes volúmenes de información provenientes de distintas fuentes, como sensores, dispositivos IoT y plataformas de monitoreo urbano. Estos datos son útiles para la gestión municipal, por ejemplo, en calidad del aire, condiciones climáticas y tráfico vehicular. El reto consiste en gestionar información con estructuras y necesidades diferentes mediante tecnologías de almacenamiento complementarias.

Por un lado, existen datos estructurados y relacionales, como ubicaciones, usuarios y organismos responsables, que requieren consistencia e integridad referencial. Por otro, las lecturas y los eventos pueden contener atributos variables y estructuras anidadas. Para abordar esta problemática, el primer sprint implementa una arquitectura híbrida: PostgreSQL organiza la información mediante un modelo normalizado hasta la tercera forma normal, mientras que MongoDB permite trabajar con documentos BSON y Aggregation Framework.

2. Fuentes de datos

Tras evaluar distintas opciones, se seleccionaron los datos abiertos del Ayuntamiento de Madrid. La generación v2 validada contiene **748.889 lecturas de enero de 2025**: 91.959 de calidad del aire y 656.930 de tráfico vehicular. Meteorología fue investigada, pero no forma parte del lote aceptado. El tráfico corresponde a una selección de puntos de medida, no a todo el conjunto de Madrid.

Como complemento, se utilizan 14.388 accidentes de 2026, agrupados por expediente y anonimizados en la salida preparada. Estos incidentes se mantienen separados de las lecturas de 2025 y no se suman a su volumen.

Las fuentes oficiales omiten algunos atributos administrativos solicitados por el proyecto. Por ello se incorporan 5.133 sensores, 5.133 usuarios y un organismo **sintéticos**, generados de forma determinista y con su procedencia identificada. Estos datos permiten demostrar las entidades del modelo, pero no acreditan dispositivos físicos, instalaciones ni responsables municipales reales. Las mediciones observadas no se modifican para completar esos atributos.

3. Arquitectura

La arquitectura de SmartCity Analytics se concibe bajo un enfoque híbrido y modular para atender las diferencias entre datos relacionales y documentales.

3.1. Enfoque híbrido de persistencia

- **Capa relacional:** PostgreSQL almacena catálogos, entidades administrativas y observaciones mediante catorce tablas normalizadas. Las claves, restricciones y triggers comprueban las relaciones del modelo.
- **Capa documental:** MongoDB almacena lecturas, incidentes, eventos derivados, alertas y entidades administrativas. Los documentos conservan metadatos de calidad, fechas y procedencia.

3.2. Flujo de integración e ingestión

1. **Extracción y normalización:** Se procesan archivos históricos para revisar fechas, unidades y referencias a los originales mediante SHA-256.
2. **Carga y control de integridad:** Se cargan las lecturas aceptadas y se comprueba que repetir el procesamiento no duplique ni altere su contenido.
3. **Consulta y verificación:** Se ejecutan las consultas de ambos motores y se contrastan los resultados con cálculos independientes en Python.

Ambos motores reciben la misma generación preparada; no se implementa una sincronización continua ni una transacción distribuida entre ellos.

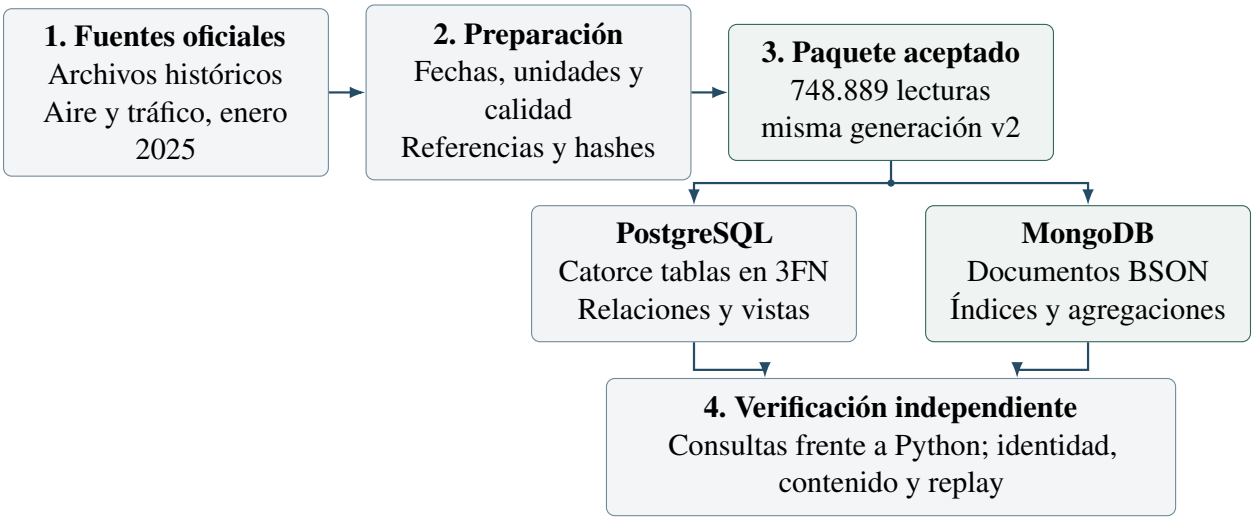

Figura 1: Flujo del lote observado. Los dos motores reciben el mismo paquete; las flechas no representan sincronización entre bases ni procesamiento distribuido.

4. Justificación tecnológica

4.1. PostgreSQL y tercera forma normal

PostgreSQL permite representar relaciones mediante claves primarias y foráneas, evitando referencias inconsistentes. La normalización separa información de fuentes, estaciones, tipos de dispositivos y observaciones para reducir redundancias y anomalías de actualización. Las consultas SQL permiten relacionar estos datos y obtener resultados verificables.

4.2. MongoDB

MongoDB permite conservar documentos con estructuras anidadas y atributos variables. Sus validadores establecen controles sobre las colecciones principales, mientras que Aggregation Framework facilita filtros, agrupaciones, conteos y promedios. Esta flexibilidad resulta útil para representar lecturas y documentos con distinta estructura, sin implicar ausencia de reglas de calidad.

4.3. Sinergia del enfoque híbrido

Ambos paradigmas permiten estudiar representaciones complementarias de los datos. La consistencia entre las lecturas se comprueba comparando identificadores y hashes de contenido. No se realizó una comparación de rendimiento que permita declarar un motor superior al otro.

5. Modelo relacional

La implementación PostgreSQL se diseñó bajo un modelo de catorce tablas normalizado hasta 3FN respecto de las dependencias funcionales documentadas. Los campos JSONB se consideran dominios SQL indivisibles; no se afirma normalización recursiva de sus propiedades. El modelo organiza la información de forma que las observaciones no repitan atributos determinados por los catálogos.

Se distingue entre estación, canal lógico y sensor físico. Una estación identifica un sitio de la fuente; un canal representa una serie de una magnitud; una observación registra un valor en un instante. La relación entre canal y sensor solo se incorpora cuando se conoce. En las lecturas públicas actuales no existen asignaciones físicas acreditadas.

El esquema contiene diecisiete claves foráneas. Cada referencia obligatoria exige un padre y las opcionales permiten su ausencia. Un padre puede tener varias filas hijas, salvo la asignación de canal a sensor, limitada a una fila por canal. Las vistas reconstruyen los atributos compartidos sin duplicarlos en las tablas. El DDL completo, con atributos y restricciones, está en `sprint1/postgres/schema_v2_3nf.sql`.

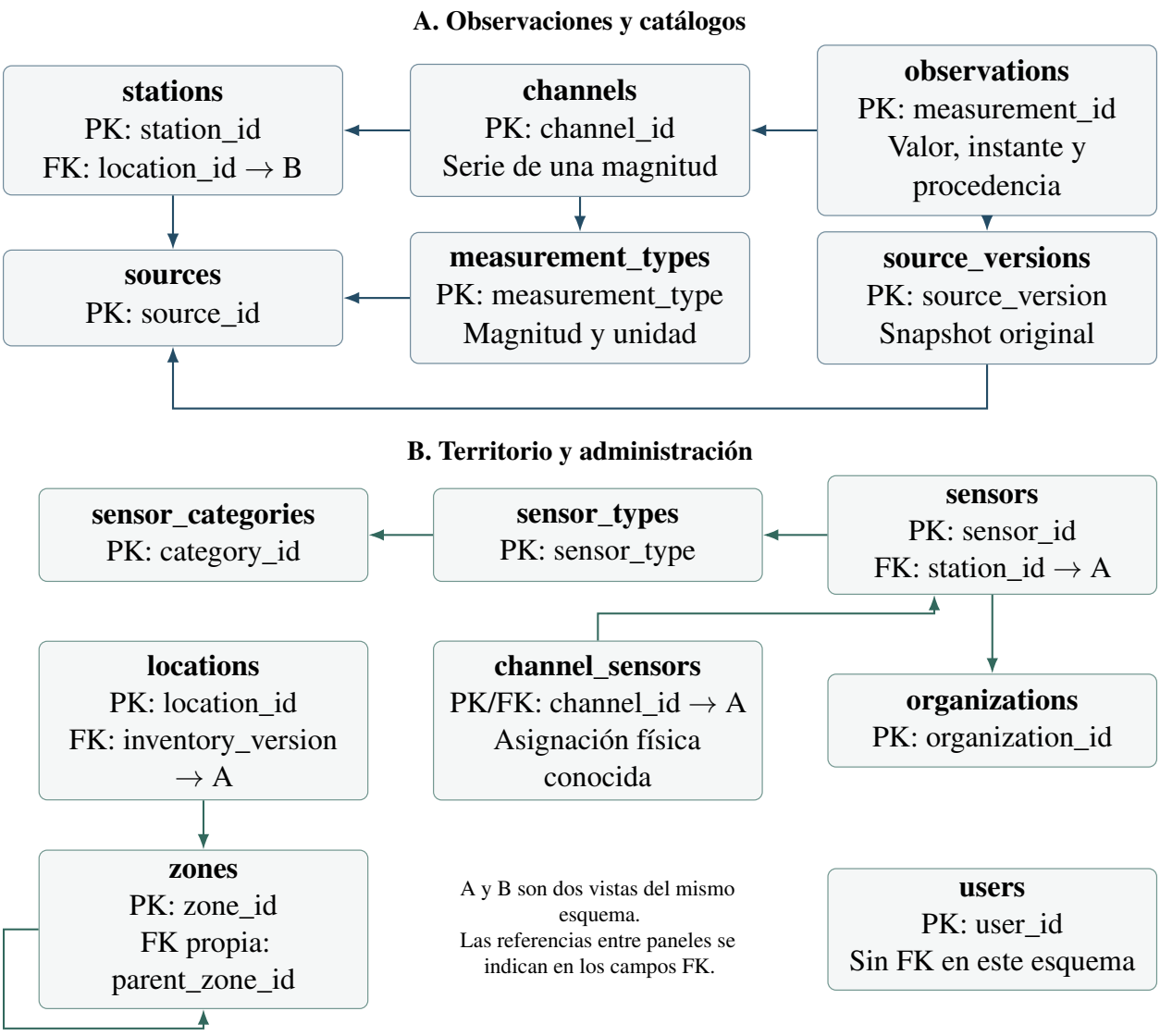

Figura 2: Las catorce tablas, agrupadas para facilitar la lectura. Las flechas apuntan de hija a padre; las cuatro FK entre paneles se indican dentro de las cajas. PK: clave primaria. FK: clave foránea.

6. Modelo documental

Para gestionar la información semiestructurada se implementa un entorno documental basado en MongoDB. La generación v2 utiliza siete colecciones: `readings`, `derived_events`, `alerts`, `incidents`, `organizations`, `users` y `sensors`. Las lecturas incluyen valor, unidad, calidad, fecha BSON y referencias al origen.

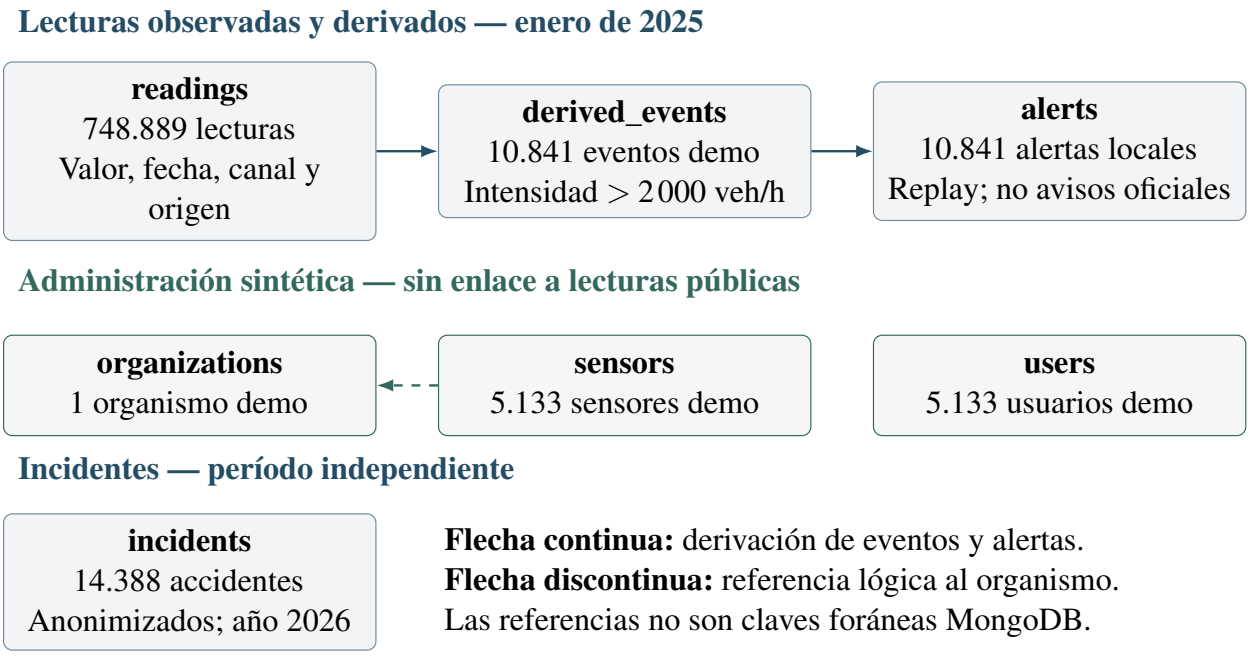

Figura 3: Las siete colecciones, separadas por origen y período. Las cuatro cajas azules tienen validador; las tres administrativas no. El laboratorio sintético se mantiene en otra base.

6.1. Validación e integridad

El cargador `sprint1/basico/cargar_mongodb_v2.js` define validadores para lecturas, incidentes, eventos y alertas. Las tres colecciones administrativas conservan su procedencia sintética, pero no tienen validadores adicionales. La identidad de cada lectura y el hash de su contenido permiten comprobar que el replay mantenga los documentos sin sobrescribir cambios incompatibles.

6.2. Indexación y consultas

Se registraron trece índices entre las siete colecciones. Las lecturas incorporan un índice único por canal, fecha y rol temporal, además de índices compuestos por fuente, magnitud y fecha, y por zona y magnitud. Los eventos, alertas e incidentes tienen índices orientados a sus consultas. Se capturaron planes de ejecución, sin afirmar que todos los filtros estén completamente optimizados.

El núcleo complementario de práctica utiliza `stageReading`, `staging_key` y cuarentena para demostrar duplicados y conflictos con ejemplos acotados. Es una implementación distinta de la carga masiva v2. Un laboratorio sintético separado demuestra consultas por sensor, por tipo de dispositivo y recuperación de texto narrativo con atributos variables y revisiones.

7. Estrategia Big Data

La estrategia de manejo de volumen se fundamenta en cargas históricas por lotes, trazabilidad e índices para apoyar las consultas. Las 748.889 lecturas aceptadas superan los objetivos de 100.000 registros PostgreSQL y 500.000 documentos MongoDB. Los catálogos, replay, incidentes y datos sintéticos no se suman a ese volumen.

MongoDB carga lotes de 1.000 documentos para evitar mantener todo el archivo en memoria. Las agregaciones permiten producir métricas y eventos derivados sin mezclar unidades ni fuentes. Los hashes ayudan a verificar que ambos motores conserven las mismas identidades y contenidos.

El alcance implementado no incluye recepción en tiempo real ni almacenamiento distribuido. MongoDB se probó en modo standalone: no se demostraron replica set ni sharding. Hadoop/HDFS y Spark corresponden a la continuación del proyecto. El requisito de almacenamiento distribuido MongoDB sigue abierto y su modalidad suficiente debe aclararse con el docente. Tampoco se implementaron vistas materializadas ni particionamiento PostgreSQL; su exigencia para este avance permanece por confirmar. Un índice parcial no equivale a particionamiento.

8. Procesamiento

8.1. Preparación y capa relacional

El módulo `preparar_v2.py` revisa los archivos originales, estandariza fechas y unidades y aplica reglas de calidad por fuente. Los rechazos se registran por su motivo; los valores ausentes no se convierten en cero ni se inventan atributos físicos. Los accidentes se deduplican por expediente y se anonimizan en la salida, manteniendo separado su período de 2026.

En PostgreSQL, el ejecutor aplica el modelo normalizado en una instancia temporal y carga catálogos y observaciones. La vista `observation_contract` reconstruye la representación compartida. Se compara con los documentos de entrada para comprobar que la normalización conserva identificadores, valores, fechas y metadatos. Las consultas se contrastan con Python.

8.2. Pipeline documental

La carga v2 de MongoDB inserta lecturas por lotes y comprueba conteos y replay. Después deriva eventos de tráfico con intensidad superior a 2.000 veh/h y genera alertas locales para zonas conocidas. La regla es de demostración: no se presenta como umbral legal ni como aviso oficial emitido en 2025.

Las consultas emplean filtros, proyecciones, ordenamiento y Aggregation Framework. Se calculan cantidades, extremos y promedios por grupos compatibles de fuente, magnitud y unidad. Los resultados se comparan con cálculos independientes sobre los archivos preparados. Las pruebas temporales no acreditan el estado de un servidor compartido ni una operación de producción.

9. Resultados

9.1. Modelo relacional

La carga normalizada PostgreSQL terminó en `PASS`, con **748.889 lecturas aceptadas**. Se registraron catorce tablas, diecisiete FK y veinticinco índices. La reconstrucción contractual presentó cero diferencias respecto a la entrada y el replay conservó el lote.

Las diez consultas `PG01-PG10` se compararon completamente con Python:

- **Catálogos e inventario:** PG01 produjo doce grupos de tipos administrativos de dispositivos, no doce magnitudes de aire. PG02 y PG05 produjeron 5.133 filas; PG04, un organismo demo; PG08, veintiún grupos de zona.
- **Estados y fechas:** PG03 produjo dos estados administrativos y PG06 cinco grupos anuales de instalación, con datos sintéticos identificados.
- **Actividad, vista y función:** PG09 produjo 360 canales y PG10 5.133 filas de conteo por estación. PG07 devolvió cero: al no existir vínculos físicos acreditados sensor–canal, ese resultado no demuestra actividad ni inactividad de dispositivos. Su lógica se comprobó con casos controlados.

9.2. Modelo documental

La ejecución masiva MongoDB también terminó en `PASS`, con 748.889 lecturas: 91.959 de `MAD-AIR` y 656.930 de `MAD-TRAFFIC`. Se verificaron las diez consultas, el replay, siete operaciones CRUD y la reconciliación de identificadores y hashes con PostgreSQL.

- **Consultas y derivados:** MG02 encontró 748.765 lecturas dentro de su ventana temporal; las otras 124 quedan fuera del filtro, no son rechazos. Se generaron 10.841 eventos y 10.841 alertas demo, además de almacenar por separado 14.388 incidentes anonimizados de 2026.
- **CRUD y laboratorio:** Las siete operaciones se ejecutaron en una colección de práctica aislada sin modificar las lecturas fuente. El laboratorio sintético verificó dieciocho documentos y trece rechazos, con consultas por sensor, por tipo y por texto narrativo.
- **Rendimiento observado:** MG10 examinó 656.930 documentos y registró 6.470 ms en la ejecución conservada. Esto no demuestra optimización completa ni permite afirmar superioridad de rendimiento entre motores.

Los resultados masivos corresponden a ejecuciones identificadas por hashes y versiones. La suite offline posterior aprobó 66 pruebas Python y 19 Node. La integración de práctica aprobó respaldo/restauración en ambos motores y una carrera PostgreSQL de ocho conexiones. Estas pruebas acotadas no sustituyen las evidencias masivas ni demuestran concurrencia de producción.

10. Conclusiones

1. **Complementariedad del modelo híbrido:** PostgreSQL permitió organizar las relaciones y restricciones del modelo normalizado, mientras que MongoDB permitió consultar documentos con estructuras variables. La comparación de identidades y hashes confirmó la consistencia del lote observado entre ambos motores.

2. **Trazabilidad y control:** Las referencias a originales, validaciones y comprobaciones de replay permiten detectar diferencias y preservar el contenido aceptado. Los datos administrativos, el laboratorio sintético y los eventos derivados se identifican por separado para no atribuirles carácter de mediciones o avisos oficiales.

3. **Alcance del avance:** Se cuenta con evidencia técnica de volumen, consultas, CRUD e integridad dentro de los entornos probados. Persisten límites de identidad física, vigencia geográfica y optimización. El almacenamiento distribuido MongoDB permanece sin demostración, por lo que los resultados no equivalen a aceptación académica global del avance.

Fuentes y evidencias de consulta

- Ayuntamiento de Madrid: calidad del aire, tráfico y accidentes. `https://datos.madrid.es/`
- Código, modelos, consultas y documentación: `https://github.com/Alpizar28/SmartCity`
- Evidencia PostgreSQL normalizada: `sprint1/postgres/evidencias/3nf/20261028T071812-55791345.json`. Evidencia MongoDB masiva: `research/datasets/results/expanded-v2-2025-01/execution-d26e40706dcd.json`. Los cierres de cada motor describen sus resultados y limitaciones.